Design of a vending machine

Name		**Class**	
Date		**Score**	
Examiner			

Contents

1 Introduction **3**

2 Experiment procedure **3**

2.1 Set Up the Project 3

2.2 Design the Components 3

2.3 Testbench Design 8

2.4 Simulation 11

3 Experimental record **12**

4 Analysis and discussion **12**

4.1 Overall Conclusion 12

4.2 Function of the System 13

4.3 Waveform Analysis 13

1 Introduction

A 4-bit vending machine system is designed to manage the sale of items with a price of 5 yuan. The system employs a finite state machine architecture to handle different payment scenarios, including accepting 5-yuan notes directly or four 1-yuan coins. This hierarchical design allows for efficient and modular implementation of state-based control on FPGA platforms. The experiment focuses on implementing and simulating this vending machine system using VHDL on Quartus II and ModelSim tools.

2 Experiment procedure

2.1 Set Up the Project

(1) Launch Quartus II 13.0 software.

(2) Open Quartus II software and create a new project using the "New Project Wizard." Name the project as vending_machine and configure the project folder and top-level entity details.

(3) For device selection, if only simulation is required, allow Quartus II to auto-select the default FPGA device.

2.2 Design the Components

The vending machine system consists of a finite state machine that manages payment processing and item dispensing based on customer input. The design follows a state-based approach to handle various payment scenarios.

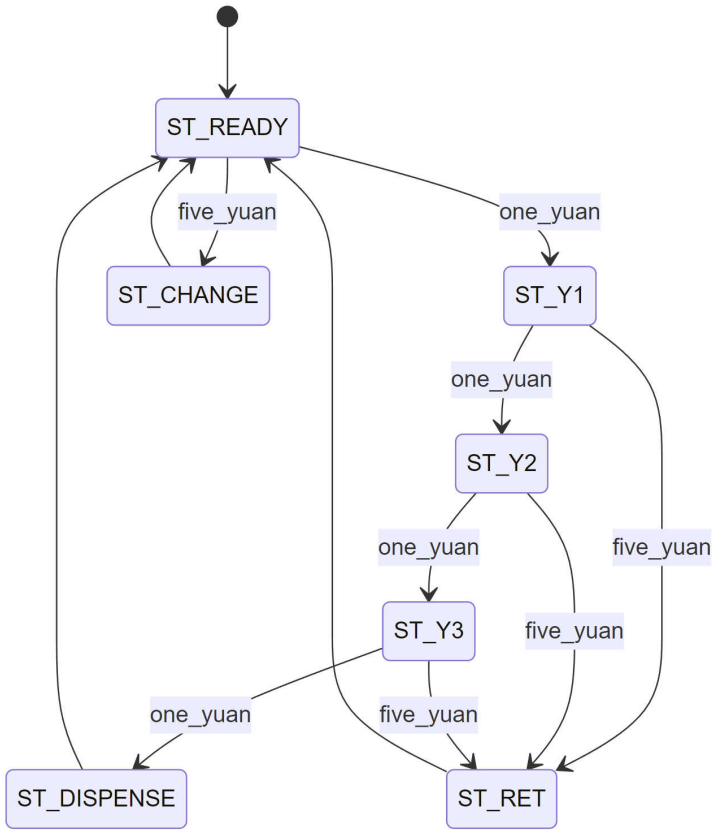

Figure 1: State diagram of the vending machine

We follow these steps to implement the vending machine:

```
library ieee;
use ieee.std_logic_1164.all;

entity vending_machine is
    port(
        clk           : in  std_logic;
        rst           : in  std_logic;
        one_yuan      : in  std_logic;
        five_yuan     : in  std_logic;
```

```
        ready         : out std_logic;
        cash          : out std_logic;
        output_ticket : out std_logic;
        give_charge   : out std_logic;
        retmoney      : out std_logic
    );
end vending_machine;

architecture fsm of vending_machine is

    type state_type is (
        ST_READY, ST_Y1, ST_Y2, ST_Y3,
        ST_DISPENSE, ST_CHANGE, ST_RET
    );

    signal state, next_state : state_type;

begin

    process(clk, rst)
    begin
        if rst = '1' then
            state <= ST_READY;
        elsif rising_edge(clk) then
            state <= next_state;
        end if;
    end process;

    process(state, one_yuan, five_yuan)
    begin
        next_state <= state;

        case state is
```

```
            when ST_READY =>
                if five_yuan = '1' then
                    next_state <= ST_CHANGE;
                elsif one_yuan = '1' then
                    next_state <= ST_Y1;
                end if;

            when ST_Y1 =>
                if five_yuan = '1' then
                    next_state <= ST_RET;
                elsif one_yuan = '1' then
                    next_state <= ST_Y2;
                end if;

            when ST_Y2 =>
                if five_yuan = '1' then
                    next_state <= ST_RET;
                elsif one_yuan = '1' then
                    next_state <= ST_Y3;
                end if;

            when ST_Y3 =>
                if five_yuan = '1' then
                    next_state <= ST_RET;
                elsif one_yuan = '1' then
                    next_state <= ST_DISPENSE;
                end if;

            when ST_DISPENSE =>
                next_state <= ST_READY;

            when ST_CHANGE =>
                next_state <= ST_READY;
```

```

            when ST_RET =>
                next_state <= ST_READY;

            when others =>
                next_state <= ST_READY;

        end case;
    end process;

    process(state)
    begin
        ready         <= '0';
        cash          <= '0';
        output_ticket <= '0';
        give_charge   <= '0';
        retmoney      <= '0';

        case state is
            when ST_READY =>
                ready <= '1';

            when ST_Y1 | ST_Y2 | ST_Y3 =>
                cash <= '1';

            when ST_DISPENSE =>
                output_ticket <= '1';

            when ST_CHANGE =>
                output_ticket <= '1';
                give_charge   <= '1';

            when ST_RET =>
                retmoney <= '1';
```

```
            when others =>
                null;
        end case;
    end process;

end fsm;
```

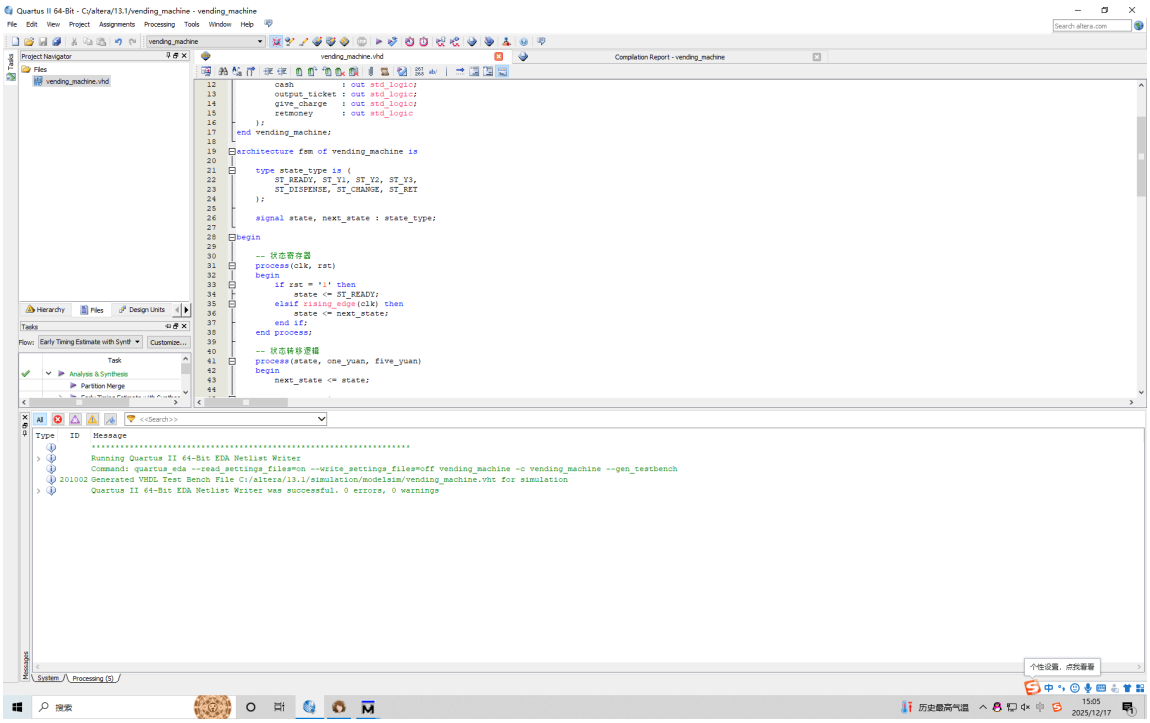

Figure 2: Vending Machine FSM Implementation

2.3 Testbench Design

(1). Create a testbench file named vending_machine_testbench.vhd.

(2). Design the testbench to test the following scenarios:

(a) Give the machine a 5-yuan cash when it is ready.

(b) Give the machine four 1-yuan cash when it is ready.

(c) Give the machine two 1-yuan cash and then a 5-yuan cash when it is ready.

(3). Add appropriate clock and signal initialization to simulate the system's behavior.

(4). Compile the testbench to verify its functionality.

```
library ieee;
use ieee.std_logic_1164.all;

entity vending_machine_tb is
end vending_machine_tb;

architecture sim of vending_machine_tb is

    signal clk           : std_logic := '0';
    signal rst           : std_logic := '1';
    signal one_yuan      : std_logic := '0';
    signal five_yuan     : std_logic := '0';

    signal ready         : std_logic;
    signal cash          : std_logic;
    signal output_ticket : std_logic;
    signal give_charge   : std_logic;
    signal retmoney      : std_logic;

begin

    uut: entity work.vending_machine
        port map(
            clk           => clk,
            rst           => rst,
            one_yuan      => one_yuan,
            five_yuan     => five_yuan,
            ready         => ready,
            cash          => cash,
```

```
            output_ticket => output_ticket,
            give_charge   => give_charge,
            retmoney      => retmoney
        );

    clk <= not clk after 5 ns;

    process
    begin
        -- reset
        rst <= '1';
        wait for 20 ns;
        rst <= '0';

        wait for 20 ns;
        five_yuan <= '1';
        wait for 10 ns;
        five_yuan <= '0';

        wait for 40 ns;

        one_yuan <= '1'; wait for 10 ns; one_yuan <= '0';
        wait for 10 ns;
        one_yuan <= '1'; wait for 10 ns; one_yuan <= '0';
        wait for 10 ns;
        one_yuan <= '1'; wait for 10 ns; one_yuan <= '0';
        wait for 10 ns;
        one_yuan <= '1'; wait for 10 ns; one_yuan <= '0';

        wait for 40 ns;

        one_yuan <= '1'; wait for 10 ns; one_yuan <= '0';
        wait for 10 ns;
        one_yuan <= '1'; wait for 10 ns; one_yuan <= '0';
```

```
        wait for 10 ns;

        five_yuan <= '1'; wait for 10 ns; five_yuan <= '0';

        wait for 50 ns;
        wait;
    end process;

end sim;
```

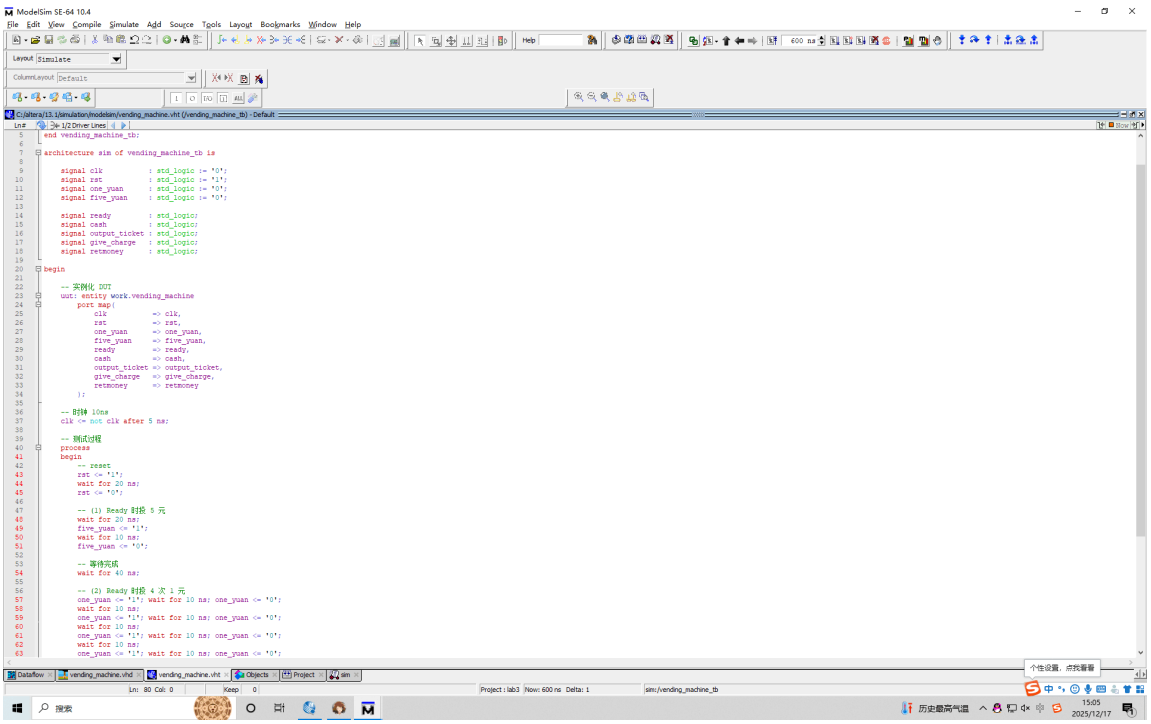

Figure 3: Testbench Code

2.4 Simulation

(1). Open ModelSim and set up the simulation environment.

(2). Add the compiled vending machine system and its testbench files to the ModelSim project.

(3). Simulate the design by applying all test cases through the testbench. Observe the output waveforms for each input combination.

(4). Record the simulation waveforms and save the results for inclusion in the report.

3 Experimental record

Below are some experimental records from the vending machine simulation.

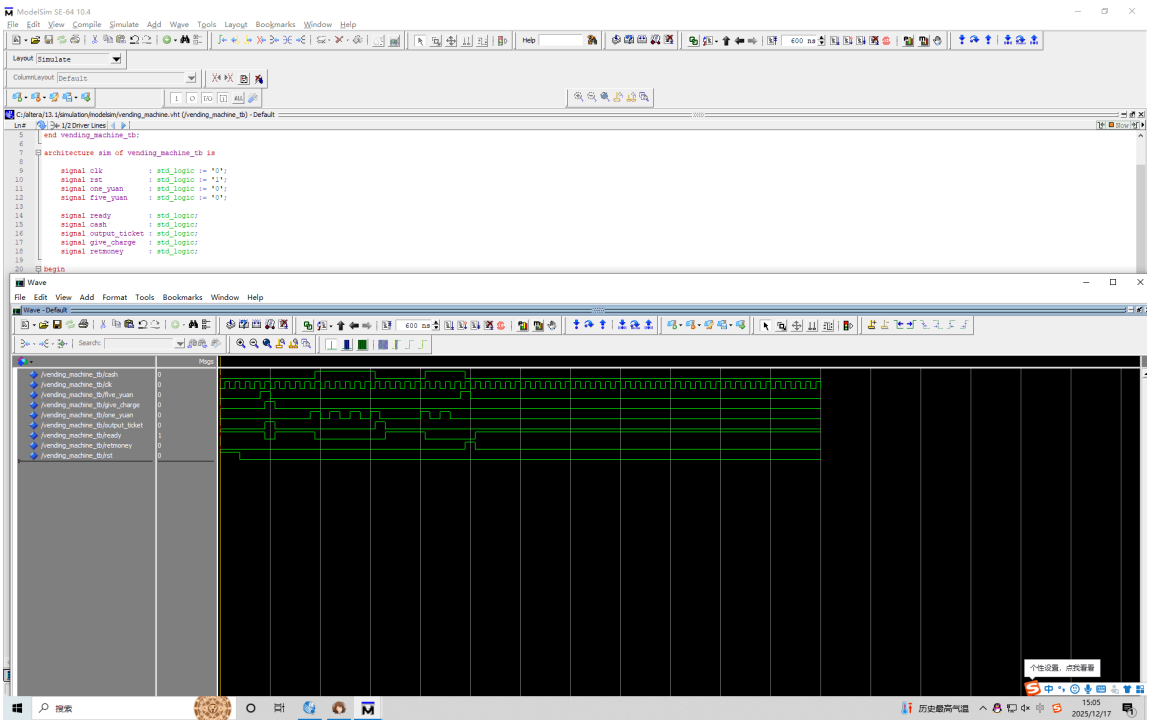

Figure 4: Simulation Waveform

4 Analysis and discussion

4.1 Overall Conclusion

The simulation waveform confirms the correct operation of the vending machine's finite state machine. Key observations include:

(1). The system correctly initializes to the **ST_READY** state upon receiving a high `rst` signal.

(2). The state transitions respond accurately to the input signals `one_yuan` and `five_yuan`:

- A 1-yuan input (`one_yuan = 1`) transitions the system through the states `ST_Y1`, `ST_Y2`, and `ST_Y3`, where four consecutive inputs are required to dispense a ticket.
- A 5-yuan input (`five_yuan = 1`) in the ready state transitions the system to `ST_CHANGE`, where a ticket is dispensed and change is given.

(3). The output signals `output_ticket`, `give_charge`, and `retmoney` are asserted at the correct times, matching the expected behavior for each state.

(4). The `retmoney` signal is asserted in the **ST_RET** state, ensuring the system handles invalid input conditions gracefully (e.g., inserting a 5-yuan input while waiting for consecutive 1-yuan inputs).

In general, the finite state machine design operates as intended, validating the logical flow of states and the corresponding outputs. The system successfully meets the functional requirements for a simple vending machine.

4.2 Function of the System

(1). The **ST_READY** state waits for customer input and outputs the `ready` signal.

(2). The **ST_Y1**, **ST_Y2**, and **ST_Y3** states accumulate consecutive 1-yuan inputs, requiring four such inputs to dispense a ticket.

(3). The **ST_CHANGE** state processes a 5-yuan payment by dispensing a ticket and providing change.

(4). The **ST_DISPENSE** state outputs the `output_ticket` signal to dispense the ticket.

(5). The **ST_RET** state returns all inserted money and then transitions back to the **ST_READY** state.

4.3 Waveform Analysis

The simulation waveform demonstrates the behavior of the vending machine under various input conditions:

(1). **Inserting 5-yuan:**

- When `five_yuan = 1` in the **ST_READY** state, the system transitions to **ST_CHANGE**.
- The `give_charge` signal is asserted, indicating that change is being dispensed.
- The `output_ticket` signal is asserted, indicating that a ticket is issued.
- After completing these actions, the system returns to the **ST_READY** state.

(2). **Inserting four consecutive 1-yuan inputs:**

- When `one_yuan = 1`, the `cash` signal remains high, indicating that the system is accumulating inserted cash.
- After four consecutive 1-yuan inputs, the state transitions to **ST_DISPENSE**.
- The `output_ticket` signal is asserted, indicating that a ticket is issued.
- The system then returns to the **ST_READY** state.

(3). **Inserting two 1-yuan inputs followed by an incorrect 5-yuan input:**

- The system initially transitions through **ST_Y1** and **ST_Y2** as two 1-yuan inputs are received.
- When a 5-yuan input (`five_yuan = 1`) is detected during this process, the system identifies an invalid input sequence.
- The state transitions to **ST_RET**, and the `retmoney` signal is asserted, indicating that all inserted money is being refunded.
- The system then returns to the **ST_READY** state.